

CS 440: Introduction to Artificial Intelligence

Assignment 2 — Search Problems in AI

Sadhana Vasanthakumar (RUID: 233000133, NetID: sv903)

Sonakshi Sharma (RUID: 229005962, NetID: ss4910)

Simone S Chanekar (RUID: 217006428, NetID: ssc183)

Due: March 11, 2026

Contents

1 Problem: A* Search from Lugoj to Bucharest	2
2 Problem: State Space Search	4
3 Problem: True or False Statements	5
4 Problem: Iterative Deepening vs. BFS	6
5 Problem: Consistent Implies Admissible; Admissible Does Not Imply Consistent	7
6 Problem: CSP Variable and Value Ordering	8
7 Problem: Minimax and Alpha-Beta Pruning	9
8 Problem: Admissibility and Consistency of Combined Heuristics	12
9 Problem: Simulated Annealing	13

1 Problem: A* Search from Lugoj to Bucharest

Setup

We trace the **tree version** of A* search, which means we do not keep a closed list. As a result, the same city can be re-expanded if it is reached again via a different path. The evaluation function is:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the actual road distance traveled from Lugoj to city n , and $h(n)$ is the straight-line distance (SLD) from n to Bucharest, which serves as our heuristic. At each step, we expand the node in the frontier with the lowest f value.

The road map is shown below.

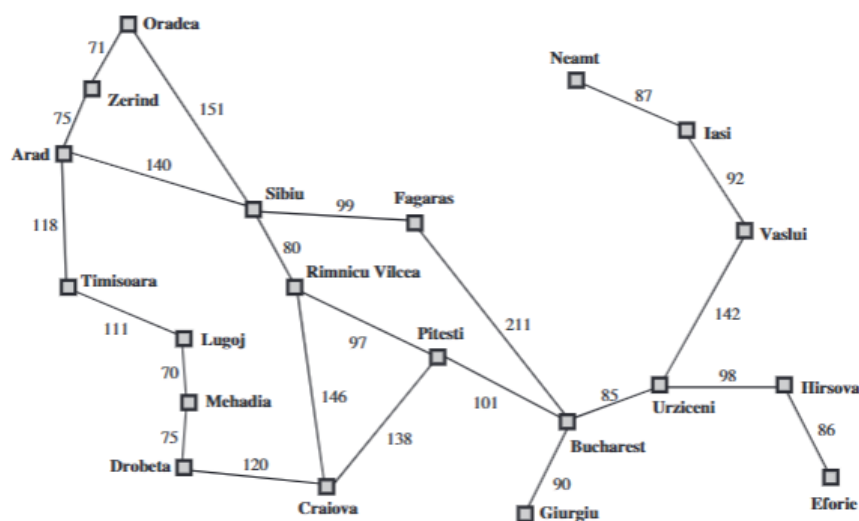


Figure 1: Road map of part of Romania with distances between cities.

Straight-line distances to Bucharest (heuristic values):

City	$h(n)$	City	$h(n)$
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Road distances used in this trace: Lugoj to Timisoara: 111; Lugoj to Mehadia: 70; Mehadia to Drobeta: 75; Drobeta to Craiova: 120; Craiova to Pitesti: 138; Craiova to Rimnicu Vilcea: 146; Timisoara to Arad: 118; Pitesti to Bucharest: 101.

Expansion Sequence

We expand one node per step, always choosing the node with the lowest f value from the frontier. Since this is the tree version, Lugoj and Mehadia appear multiple times because the algorithm re-explores them via different paths. Each entry is recorded as (city, f , g , h).

Step	City	f	g	h	What happens
1	Lugoj	244	0	244	Start. Add Timisoara ($f=440$) and Mehadia ($f=311$) to frontier.
2	Mehadia	311	70	241	Lowest f in frontier. Add Lugoj ($f=384$, via Lug→Meh) and Drobeta ($f=387$).
3	Lugoj	384	140	244	Re-expanded: tree search has no closed list, so Lugoj→Mehadia→Lugoj is a valid new frontier node with higher g . Add Timisoara ($f=580$) and Mehadia ($f=451$).
4	Drobeta	387	145	242	Add Mehadia ($f=461$, $g=220$) and Craiova ($f=425$).
5	Craiova	425	265	160	Add Pitesti ($f=503$), Rimnicu Vilcea ($f=604$), Drobeta ($f=627$).
6	Timisoara	440	111	329	Original direct path from Lugoj. Add Arad ($f=595$) and Lugoj ($f=466$).
7	Mehadia	451	210	241	Re-expanded via Lugoj→Mehadia→Lugoj→Mehadia (added in Step 3). Successors: Lugoj ($g=280$, $f=524$) and Drobeta ($g=285$, $f=527$). Both $f > 461$; neither is selected next.
8	Mehadia	461	220	241	Re-expanded via Lugoj→Mehadia→Drobeta→Mehadia (added in Step 4). Successors: Lugoj ($g=290$, $f=534$) and Drobeta ($g=295$, $f=537$). Both $f > 466$; neither is selected next.
9	Lugoj	466	222	244	Re-expanded via Lugoj→Timisoara→Lugoj (added in Step 6). Successors: Timisoara ($g=333$, $f=662$) and Mehadia ($g=292$, $f=533$). Both $f > 503$; neither is selected next.
10	Pitesti	503	403	100	Add Bucharest ($f=504$) and Rimnicu Vilcea ($f=693$).
11	Bucharest	504	504	0	Goal reached. Algorithm terminates.

Optimal path found:

Lugoj → Mehadia → Drobeta → Craiova → Pitesti → Bucharest

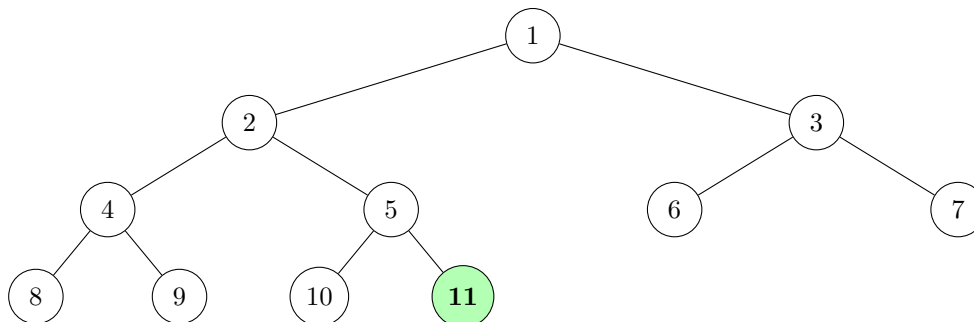
Total cost: $70 + 75 + 120 + 138 + 101 = \mathbf{504}$ km.

Note: Because this is the tree version with no closed list, Lugoj and Mehadia are re-expanded when found on new paths. A graph version would use a closed list to avoid this, making the search more efficient.

2 Problem: State Space Search

The state space begins at node 1. Each state k has two successors: $2k$ (left child) and $2k + 1$ (right child). This creates an infinite binary tree. The goal is node **11**, which lives at depth 3 along the path $1 \rightarrow 2 \rightarrow 5 \rightarrow 11$.

The first four levels of the tree are shown below.



(a) Goal state = 11

Breadth-First Search (BFS). BFS visits all nodes level by level before going deeper. It finds node 11 at depth 3 after visiting all shallower nodes first:

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, **11**

Depth-Limited Search (DLS, limit = 3). DLS runs depth-first but stops expanding any node beyond the depth limit. Node 11 is at depth 3, so it is reachable. The search goes as deep as possible on the left before backtracking:

1, 2, 4, 8, 9, 5, 10, **11**

Iterative Deepening Search (IDS). IDS runs DLS repeatedly, increasing the depth limit by 1 each time until the goal is found:

- **Limit 0:** 1
- **Limit 1:** 1, 2, 3
- **Limit 2:** 1, 2, 4, 5, 3, 6, 7
- **Limit 3:** 1, 2, 4, 8, 9, 5, 10, **11** (goal found)

The complete ordering across all iterations is:

$\underbrace{1}_{\text{limit 0}}, \underbrace{1, 2, 3}_{\text{limit 1}}, \underbrace{1, 2, 4, 5, 3, 6, 7}_{\text{limit 2}}, \underbrace{1, 2, 4, 8, 9, 5, 10, 11}_{\text{limit 3}}$

(b) Bidirectional Search

Bidirectional search runs two BFS frontiers simultaneously: one forward from node 1 and one backward from goal node 11. We alternate expanding each frontier by one level and check *after each expansion* whether the newly generated nodes intersect the opposite frontier.

Branching factors:

- **Forward (from 1):** Each node k has two children, $2k$ and $2k + 1$, so the forward branching factor is **2**.
- **Backward (from 11):** Each node $k > 1$ has exactly one parent, $\lfloor k/2 \rfloor$. So the backward branching factor is **1**. The backward path from 11 is a single chain: $11 \rightarrow 5 \rightarrow 2 \rightarrow 1$.

Execution trace. We maintain a *visited* set for each direction and check for intersection after every expansion step.

Step	Forward visited	Backward visited	Intersection?
Start	{1}	{11}	None
1F: expand 1	{1, 2, 3}	{11}	Check $\{2, 3\} \cap \{11\} = \emptyset$
1B: expand 11	{1, 2, 3}	{11, 5}	Check $\{5\} \cap \{1, 2, 3\} = \emptyset$
2F: expand 2 and 3	{1, 2, 3, 4, 5, 6, 7}	{11, 5}	Check $\{4, 5, 6, 7\} \cap \{11, 5\} = \{5\}$

The intersection $\{5\}$ is detected after step 2F. Node 5 was placed into the forward visited set (by expanding node 2) and was already in the backward visited set (placed there in step 1B). The meeting node is **5**, giving the complete path:

$$1 \rightarrow 2 \rightarrow \underbrace{5}_{\text{meeting node}} \rightarrow 11.$$

Bidirectional search works extremely well here. Because the backward branching factor is 1, the backward search is essentially free and just walks up one chain of parents. BFS would generate all 11 nodes before finding the goal, while bidirectional terminates as soon as the two frontiers overlap at node 5, having expanded only 4 nodes total (1, 2, 3, and 11) before detecting the meeting point. The asymmetry in branching factors makes this a best-case scenario for bidirectional search.

3 Problem: True or False Statements

- (a) **Breadth-first search is a special case of uniform-cost search.**

True. Uniform-cost search (UCS) always expands the node with the lowest accumulated path cost $g(n)$. When every edge in the graph has the same cost (say, cost = 1), then $g(n)$ equals the depth of n , and UCS expands nodes in depth order, which is exactly what BFS does. So BFS is just UCS applied to a graph with uniform edge costs.

- (b) **Depth-first search is a special case of best-first tree search.**

True. Best-first search expands the node with the lowest value of some evaluation function $f(n)$. If we set $f(n) = -\text{depth}(n)$, then the node with the lowest f value is always the deepest node currently in the frontier, which causes the search to keep diving deeper before backtracking. That is exactly the behavior of DFS.

- (c) **Uniform-cost search is a special case of A* search.**

True. A* uses $f(n) = g(n) + h(n)$. If we set $h(n) = 0$ for all nodes, then $f(n) = g(n)$, which is exactly the evaluation function used by UCS. The zero heuristic is trivially admissible since it never overestimates. So UCS is just A* with $h \equiv 0$.

- (d) **Depth-first graph search is guaranteed to return an optimal solution.**

False. DFS returns the first complete path to the goal it finds, with no regard for cost. That first path may be much more expensive than the optimal one. DFS offers no optimality guarantee.

- (e) **Breadth-first graph search is guaranteed to return an optimal solution.**

False. BFS guarantees the *shallowest* solution (fewest edges), but not the cheapest one. When edge costs are unequal, a shallower path can easily cost more than a deeper path. BFS is only cost-optimal when all edge costs are equal.

- (f) **Uniform-cost graph search is guaranteed to return an optimal solution.**

True. UCS expands nodes in order of increasing $g(n)$. When a goal node is dequeued for expansion, every remaining node in the frontier has $g \geq g(\text{goal})$, so no cheaper path to the goal can exist. This guarantee holds as long as all edge costs are non-negative.

- (g) **A* graph search is guaranteed to return an optimal solution if the heuristic is consistent.**

True. A consistent heuristic satisfies $h(n) \leq c(n, a, n') + h(n')$ for every edge. This ensures that f -values along any path are non-decreasing, so the first time A* expands a node it has found the cheapest path to it. Therefore, the first time the goal is expanded, the path is optimal.

- (h) **A* graph search is guaranteed to expand no more nodes than depth-first graph search if the heuristic is consistent.**

False. DFS can stumble on the goal very quickly if it happens to follow the right branch first, potentially expanding far fewer nodes than A*. A* provides no upper-bound guarantee on expansions relative to DFS.

- (i) **A* graph search is guaranteed to expand no more nodes than uniform-cost graph search if the heuristic is consistent.**

True. UCS is simply A* with $h \equiv 0$. A consistent heuristic $h \geq 0$ increases the f -value of every node compared to UCS, which means fewer nodes fall below the optimal cost threshold C^* and get expanded. A* therefore expands no more nodes than UCS, and typically many fewer.

4 Problem: Iterative Deepening vs. BFS

Advantage of IDS over BFS.

IDS only needs to store the current path from the root to the node being expanded, giving it a memory footprint of $O(b \cdot d)$ where b is the branching factor and d is the solution depth. BFS must

store the entire frontier, which can grow to $O(b^d)$ nodes, one for every node at the current depth level. For large search trees this difference is enormous. IDS can tackle problems where BFS would run out of memory entirely.

Disadvantage of IDS compared to BFS.

IDS re-expands every shallower node on each new iteration. By the time it reaches depth d , nodes near the root have been re-expanded roughly $b/(b-1)$ times more than they would be in BFS. BFS never re-expands any node. So while both algorithms have the same $O(b^d)$ time complexity asymptotically, IDS does more actual work by a constant factor and is therefore slower in practice for problems where memory is not a constraint.

5 Problem: Consistent Implies Admissible; Admissible Does Not Imply Consistent

Definitions

A heuristic h is **admissible** if it never overestimates the true cost to reach the goal. Formally, $h(n) \leq h^*(n)$ for all nodes n , where $h^*(n)$ is the actual cheapest cost from n to the goal.

A heuristic h is **consistent** (also called monotone) if for every node n and every action a leading to successor n' with cost $c(n, a, n')$:

$$h(n) \leq c(n, a, n') + h(n'), \quad h(\text{goal}) = 0.$$

Intuitively, the estimated cost from n cannot exceed the step cost plus the estimated cost from the next state. This is a triangle-inequality-style condition.

Proof: Consistency Implies Admissibility

Proof. We prove that $h(n) \leq h^*(n)$ for all n by induction on k , the number of steps in the optimal path from n to the goal.

Base case ($k = 0$): Node n is itself the goal. By definition, $h(\text{goal}) = 0 = h^*(\text{goal})$, so the claim holds.

Inductive step: Suppose the claim holds for all nodes whose optimal distance to the goal is at most k . Let n be a node whose optimal distance is $k+1$, and let n' be the next node on the optimal path from n , reached via action a with step cost $c(n, a, n')$. Then:

$$h^*(n) = c(n, a, n') + h^*(n').$$

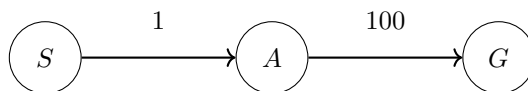
Since n' is only k steps from the goal, the inductive hypothesis gives $h(n') \leq h^*(n')$. Applying consistency to edge $n \rightarrow n'$:

$$h(n) \leq c(n, a, n') + h(n') \leq c(n, a, n') + h^*(n') = h^*(n).$$

The induction is complete. Every consistent heuristic is admissible. □

Example: A Heuristic That Is Admissible but Not Consistent

Consider this simple three-node graph, where S is the start and G is the goal:



The true optimal costs are: $h^*(G) = 0$, $h^*(A) = 100$, $h^*(S) = 1 + 100 = 101$.

Now define the heuristic: $h(G) = 0$, $h(A) = 1$, $h(S) = 100$.

Admissibility check:

- $h(G) = 0 \leq 0 = h^*(G)$ ✓
- $h(A) = 1 \leq 100 = h^*(A)$ ✓
- $h(S) = 100 \leq 101 = h^*(S)$ ✓

All values are within the true cost, so h is **admissible**.

Consistency check for edge $S \rightarrow A$ (cost 1):

$$h(S) \leq c(S, A) + h(A) \quad \implies \quad 100 \leq 1 + 1 = 2. \quad \text{False!}$$

This is a clear violation. The heuristic at S drops sharply at A , even though the actual remaining cost from A is 100. The heuristic is **admissible but not consistent**. ■

Intuition: $h(S) = 100$ is a reasonable estimate of the total cost (101), but $h(A) = 1$ drastically underestimates the remaining cost from A (which is 100). This sudden drop violates the consistency triangle inequality, even though neither value alone overestimates h^* .

6 Problem: CSP Variable and Value Ordering

In a Constraint Satisfaction Problem (CSP), we assign values to variables one at a time while satisfying a set of constraints. Two heuristics guide which variable and value to pick next, and they use opposite philosophies for good reason.

Most-Constrained Variable (Minimum Remaining Values, MRV). When choosing which variable to assign next, pick the one with the **fewest legal values remaining** in its domain.

This is a *fail-first* strategy. The most-constrained variable is the one most likely to cause a dead end soon. By assigning it now, we find out whether a failure is inevitable as early as possible in the search tree, before wasting effort assigning many other variables. Every branch pruned near the root of the tree eliminates an exponentially large subtree of useless work. Choosing the hardest variable first is simply the most efficient way to catch problems early.

Least-Constraining Value. Once a variable has been selected, choose the value that **rules out the fewest options** for other unassigned variables.

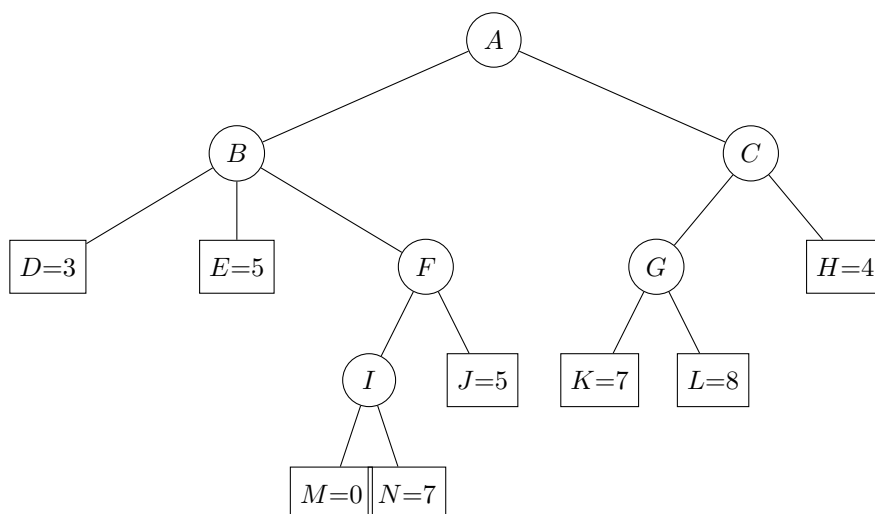
This is a *succeed-first* strategy. We are trying to find one valid complete assignment, so we want to preserve as much flexibility as possible for the variables we have not yet assigned. Picking a value that eliminates fewer options from neighboring domains gives us the best chance of completing the assignment successfully without needing to backtrack.

Why opposite strategies? For *variables*, failing fast is valuable because any dead end found high in the tree prunes away a huge number of descendants. For *values*, we want to succeed and find a solution, so maximizing future freedom is the right goal. Together, the combination prunes dead ends aggressively while keeping the best options open for completing the assignment.

7 Problem: Minimax and Alpha-Beta Pruning

Tree Structure

The game tree for this problem is shown below. MAX moves first at the root A ; MIN moves at the next level.



Circular nodes are internal (non-leaf) nodes; rectangular nodes are terminal leaf nodes with their values shown. A is a MAX node; B and C are MIN nodes; F and G are MAX nodes; I is a MIN node.

(a) Minimax Values

We compute minimax values bottom-up. MIN nodes return the minimum of their children's values; MAX nodes return the maximum.

$$\begin{aligned}
 \text{minimax}(I) &= \min(0, 7) = 0 \\
 \text{minimax}(F) &= \max(0, 5) = 5 \\
 \text{minimax}(B) &= \min(3, 5, 5) = 3 \\
 \text{minimax}(G) &= \max(7, 8) = 8 \\
 \text{minimax}(C) &= \min(8, 4) = 4 \\
 \text{minimax}(A) &= \max(3, 4) = 4
 \end{aligned}$$

The best move for MAX at node A is to go to C , which guarantees a value of 4. Going to B only guarantees a value of 3.

(b) Left-to-Right Alpha-Beta Pruning

Alpha-beta pruning skips subtrees that cannot change the final decision. We track two bounds:

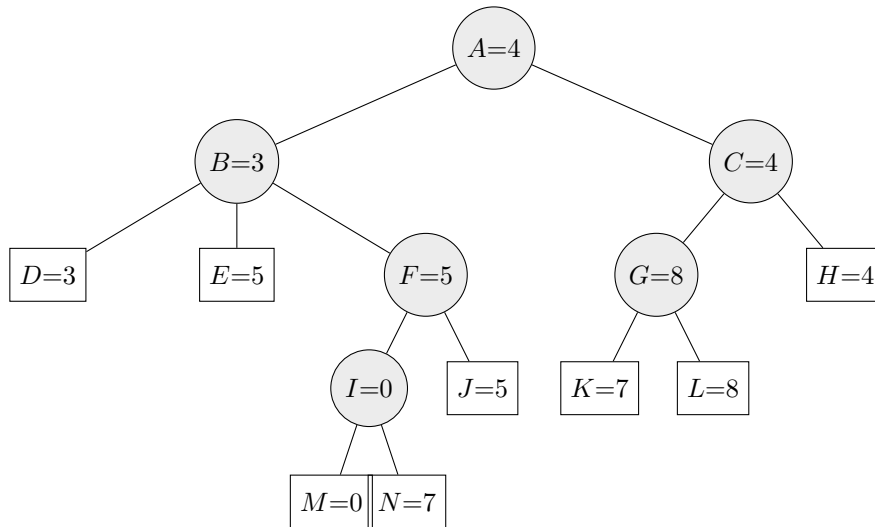
- α : the best value MAX can currently guarantee anywhere on the path to the root.
- β : the best value MIN can currently guarantee anywhere on the path to the root.

We prune a subtree whenever $\alpha \geq \beta$.

Expanding children **left before right**:

1. Visit A (MAX), $\alpha = -\infty$, $\beta = +\infty$. Recurse into B .
2. Visit B (MIN), $\alpha = -\infty$, $\beta = +\infty$.
 - $D = 3$: $\beta \leftarrow 3$. No cutoff.
 - $E = 5$: $\min(3, 5) = 3$, β stays at 3. No cutoff.
 - Recurse into F (MAX) with $\alpha = -\infty$, $\beta = 3$.
 - Recurse into I (MIN) with $\alpha = -\infty$, $\beta = 3$.
 - * $M = 0$: $\beta_I \leftarrow 0$. Check $\alpha \geq \beta_I$: $-\infty \geq 0$? No. Continue.
 - * $N = 7$: I returns $\min(0, 7) = 0$.
 - $\alpha_F \leftarrow 0$. Check $0 \geq 3$? No.
 - $J = 5$: $\alpha_F \leftarrow 5$. F returns 5. ($\alpha_F = 5 \geq \beta_B = 3$, but F is the last child of B , so no cutoff propagates upward.)
 - B returns $\min(3, 5, 5) = 3$.
3. Back at A : $\alpha \leftarrow 3$. Recurse into C (MIN) with $\alpha = 3$, $\beta = +\infty$.
 - Recurse into G (MAX) with $\alpha = 3$, $\beta = +\infty$.
 - $K = 7$: $\alpha_G \leftarrow 7$. No cutoff.
 - $L = 8$: $\alpha_G \leftarrow 8$. G returns 8.
 - $\beta_C \leftarrow 8$. Check $3 \geq 8$? No.
 - $H = 4$: $\beta_C \leftarrow 4$. Check $3 \geq 4$? No. C returns 4.
4. $A = \max(3, 4) = 4$.

Result: In the left-to-right ordering, **no nodes are pruned** in this tree. The bound of $\alpha = 3$ established by exploring B first is not tight enough to trigger a cutoff anywhere in C 's subtree.



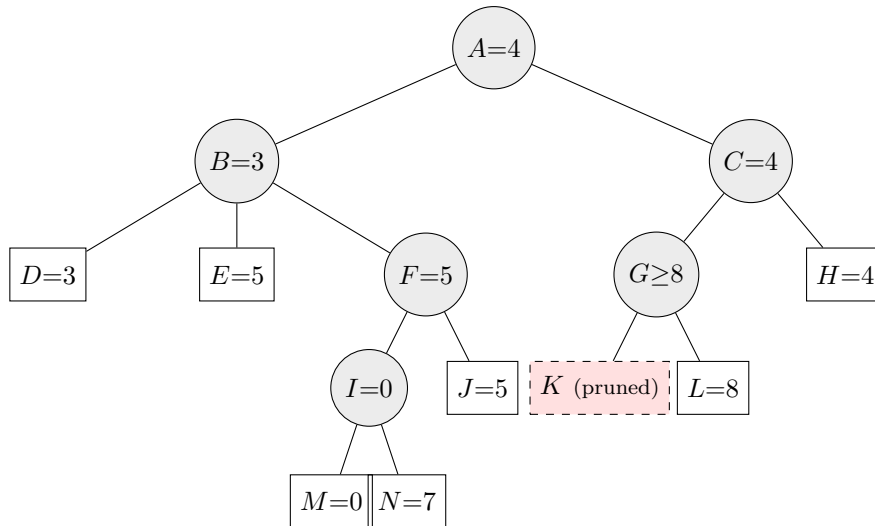
Left-to-right: all nodes are visited. No pruning occurs. Grey shaded nodes show the backed-up minimax values.

(c) Right-to-Left Alpha-Beta Pruning

Now we expand children **right before left**.

1. Visit A (MAX), $\alpha = -\infty$, $\beta = +\infty$. Recurse into C first (rightmost child).
2. Visit C (MIN), $\alpha = -\infty$, $\beta = +\infty$.
 - $H = 4$ (rightmost child): $\beta_C \leftarrow 4$. No cutoff.
 - Recurse into G (MAX) with $\alpha = -\infty$, $\beta = 4$.
 - $L = 8$ (right child first): $\alpha_G \leftarrow 8$. Check $\alpha_G \geq \beta$: $8 \geq 4$? **Yes. Beta cutoff. Node K is pruned.**
 - G returns a value ≥ 8 . $\beta_C \leftarrow \min(4, 8) = 4$. C returns 4.
3. Back at A : $\alpha \leftarrow 4$. Recurse into B (MIN) with $\alpha = 4$, $\beta = +\infty$.
 - F is the rightmost child of B . Recurse into F (MAX) with $\alpha = 4$, $\beta = +\infty$.
 - $J = 5$ (right child): $\alpha_F \leftarrow 5$. No cutoff.
 - Recurse into I (MIN) with $\alpha = 4$, $\beta = +\infty$.
 - * $N = 7$ (right child): $\beta_I \leftarrow 7$. Check $4 \geq 7$? No.
 - * $M = 0$ (left child, last): $\beta_I \leftarrow 0$. Check $\alpha \geq \beta_I$: $4 \geq 0$? Yes, but M is the last child so nothing more to prune. I returns 0.
 - $F = \max(5, 0) = 5$.
 - $\beta_B \leftarrow 5$. Check $4 \geq 5$? No.
 - $E = 5$: $\beta_B \leftarrow 5$. Check $4 \geq 5$? No.
 - $D = 3$ (leftmost, last): $\beta_B \leftarrow 3$. Check $\alpha \geq \beta_B$: $4 \geq 3$? Yes, but D is the last child so nothing further to prune. B returns 3.
4. $A = \max(4, 3) = 4$.

Result: Node K is pruned by the beta cutoff at G .



Right-to-left: node K is pruned (shown dashed in red). After seeing $L=8 \geq \beta=4$ at G , there is no need to evaluate K .

Why does different pruning occur in the two orderings? Alpha-beta pruning can only cut off a branch when the α or β bound established so far is tight enough to make exploring further pointless. Whether a bound is tight depends entirely on which subtrees have already been evaluated.

In the **left-to-right** ordering, B is explored first and returns a value of 3, setting $\alpha = 3$ at A . When we then explore C , this $\alpha = 3$ is not strong enough to trigger any cutoffs inside C .

In the **right-to-left** ordering, C is explored first. Seeing $H = 4$ immediately sets $\beta_C = 4$. When we recurse into G and see $L = 8$, we know G will return at least 8, which is already worse for MIN than the 4 from H . There is no reason to check K , so it is pruned.

The key takeaway is: **alpha-beta pruning is most effective when the best moves are explored first**, since that creates the tightest bounds the soonest. Optimal move ordering can reduce the effective branching factor significantly.

8 Problem: Admissibility and Consistency of Combined Heuristics

We are given two admissible heuristics h_1 and h_2 , so $h_i(n) \leq h^*(n)$ for all n . We are also given that both are consistent, so $h_i(n) \leq c(n, a, n') + h_i(n')$ for all edges. We analyze three new heuristics built from these.

(a) $h(n) = \min\{h_1(n), h_2(n)\}$

Admissible? Yes. Since both $h_1(n)$ and $h_2(n)$ are at most $h^*(n)$, their minimum is too:

$$h(n) = \min\{h_1(n), h_2(n)\} \leq h_1(n) \leq h^*(n). \quad \checkmark$$

Consistent? Yes. Let $h(n) = h_1(n)$ (i.e., h_1 achieves the min at n , so $h_1(n) \leq h_2(n)$). We consider two sub-cases based on which heuristic achieves the min at n' :

- **Sub-case 1:** $h(n') = h_1(n')$. Then by consistency of h_1 :

$$h(n) = h_1(n) \leq c(n, a, n') + h_1(n') = c(n, a, n') + h(n'). \quad \checkmark$$

- **Sub-case 2:** $h(n') = h_2(n') < h_1(n')$. Since $h_1(n) \leq h_2(n)$ and h_2 is consistent ($h_2(n) \leq c(n, a, n') + h_2(n')$), chaining gives:

$$h(n) = h_1(n) \leq h_2(n) \leq c(n, a, n') + h_2(n') = c(n, a, n') + h(n'). \quad \checkmark$$

Summary: admissible and consistent.

(b) $h(n) = w h_1(n) + (1-w) h_2(n), \quad 0 \leq w \leq 1$

Admissible? Yes. This is a weighted average of h_1 and h_2 . Since each is at most $h^*(n)$:

$$h(n) = w h_1(n) + (1-w) h_2(n) \leq w h^*(n) + (1-w) h^*(n) = h^*(n). \quad \checkmark$$

Consistent? Yes. Using consistency of both h_1 and h_2 :

$$\begin{aligned} h(n) &= w h_1(n) + (1-w) h_2(n) \\ &\leq w [c(n, a, n') + h_1(n')] + (1-w) [c(n, a, n') + h_2(n')] \\ &= c(n, a, n') + w h_1(n') + (1-w) h_2(n') \\ &= c(n, a, n') + h(n'). \quad \checkmark \end{aligned}$$

Summary: admissible and consistent.

(c) $h(n) = \max\{h_1(n), h_2(n)\}$

Admissible? Yes. Both $h_1(n) \leq h^*(n)$ and $h_2(n) \leq h^*(n)$, so their maximum is also at most $h^*(n)$:

$$h(n) = \max\{h_1(n), h_2(n)\} \leq h^*(n). \quad \checkmark$$

Consistent? Yes. Suppose $h(n) = h_1(n)$ (i.e., h_1 achieves the max at n). From consistency of h_1 :

$$h(n) = h_1(n) \leq c(n, a, n') + h_1(n') \leq c(n, a, n') + \max\{h_1(n'), h_2(n')\} = c(n, a, n') + h(n'). \quad \checkmark$$

Summary: admissible and consistent.

Which heuristic is best for A*?

We prefer option (c): $h(n) = \max\{h_1(n), h_2(n)\}$.

In A*, a larger admissible heuristic is always better because it raises the f -values of nodes farther from the goal, causing A* to skip over them more aggressively and expand fewer nodes overall. Formally, if $h_a(n) \geq h_b(n)$ for all n and both are admissible, then every node expanded by A* with h_a is also expanded by A* with h_b , but not vice versa — we say h_a *dominates* h_b .

Since $\max\{h_1, h_2\} \geq h_1(n)$, $\max\{h_1, h_2\} \geq h_2(n)$, and $\max\{h_1, h_2\} \geq w h_1(n) + (1-w) h_2(n)$ for all n , heuristic (c) dominates both (a) and (b). It provides the tightest admissible lower bound available from our two heuristics while remaining consistent, making it the best choice for A*.

9 Problem: Simulated Annealing

Simulated annealing (SA) extends hill climbing by occasionally accepting moves that make the objective function worse, controlled by a temperature parameter T that decreases over time. This allows the algorithm to escape local optima.

(a) When is hill climbing better than simulated annealing?

Hill climbing is better, and the random component of SA is unnecessary, when the objective function is **unimodal**: it has a single global maximum and no local optima that are not also the global optimum.

In such a landscape, every uphill move leads toward the global peak. There is no risk of getting trapped at a local maximum, so randomly accepting worse moves would only slow down convergence. Examples include convex optimization problems and any search space with a single smooth basin leading up to one peak.

(b) When does random guessing work just as well as simulated annealing?

Random guessing is just as effective as SA when the objective function's landscape has **no useful local structure**: the values of neighboring states are essentially uncorrelated (a completely random landscape).

In this case, the hill-climbing component of SA, which prefers uphill moves, provides no useful guidance at all. A locally good state is no more likely to be near the global optimum than any randomly chosen state. So random sampling of the search space performs just as well as the structured approach of SA.

(c) For what types of problems is simulated annealing most useful?

Combining the insights from (a) and (b): SA is most useful when the landscape is **multimodal but structured**. Specifically, the landscape should have:

- Many local optima, so that pure hill climbing gets trapped, and
- Enough local structure (correlation between neighboring states) that uphill moves are generally useful, so random guessing is inefficient.

The implicit assumption SA makes is that the landscape is rugged but not random: there are many hills and valleys, but downhill moves taken at the right time (when the temperature is high) can eventually lead to the global optimum. Classic examples are the Traveling Salesman Problem, protein folding, and VLSI circuit layout.

(d) A smarter use of the state values we observe

Standard SA returns the state it is currently in when the annealing schedule ends. But the algorithm may have visited a much better state earlier during the high-temperature phase and then drifted away from it through random moves.

A smarter approach is to **track the best state seen at any point during the entire run**. We maintain a variable `best_state` alongside the current state. After every single step, if the current state's value is higher than `best_state`'s value, we update `best_state`. At termination, we return `best_state` rather than the current state.

This modification costs essentially nothing in extra computation or memory, and it is guaranteed to return a result at least as good as standard SA and typically much better.

(e) Making productive use of two million states of memory

Rather than running SA sequentially a million times with independent random restarts, a much more powerful approach is **population-based parallel SA**:

1. Initialize a large population of states (say, one million), each starting from a different random position in the search space.
2. Run a separate SA chain for each state, each with its own temperature schedule.
3. Periodically **reseed the population**: identify the top-performing states across all chains, discard the weakest, and replace them with slightly perturbed copies of the best states found so far.
4. Maintain a global record of the single best state seen across all chains at all times.

This is superior to sequential restarts because information about good regions of the search space, discovered by any one chain, immediately helps seed other chains. The diverse population also prevents all chains from converging to the same local optimum, and the collective search covers the space much more efficiently.

(f) Adapting SA-style randomness to gradient ascent

Standard gradient ascent updates $x \leftarrow x + \eta \nabla f(x)$, always following the steepest uphill direction. This is efficient locally but always converges to the nearest local maximum.

To adapt the randomness of SA, we add **annealed noise** to the gradient step:

$$x \leftarrow x + \eta \nabla f(x) + \varepsilon_T,$$

where ε_T is a random vector whose magnitude is proportional to the current temperature T . We also apply the SA acceptance rule: always accept a step that increases f , but accept a step that decreases f with probability $e^{-|\Delta f|/T}$. The temperature T is decreased over time following an annealing schedule.

At high temperatures, the large noise allows the algorithm to escape local maxima by jumping to distant parts of the landscape. As T cools, the noise shrinks and the gradient becomes the dominant force, guiding the algorithm to converge precisely. The result is **stochastic gradient ascent with annealed noise**, which combines the global exploration of SA with the efficient local exploitation of gradient methods.